SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	J KARTHIK	**Reg. No.:**	192472136
Section / Batch:	CSE – AI	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

*I J **KARTHIK / 192472136** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: J KARTHIK

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1. Debugging Selection Sort Code

The given code attempts a brute-force Selection Sort but uses the wrong comparison operator when searching for the pivot element of each pass.

Buggy Code

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] > arr[min_idx]:   # ERROR
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Errors Identified & Root Cause

- Comparison operator error: 'if arr[j] > arr[min_idx]' searches for the MAXIMUM element in the unsorted sub-array, not the minimum.
- Root cause: Selection Sort must repeatedly select the smallest remaining element and move it to the front for ascending order. Using '>' instead of '<' inverts the selection logic, so the algorithm silently sorts the array in descending order instead of ascending order — it never raises an exception, which makes the bug easy to miss during casual testing.

Step-by-Step Debugging

- Trace arr = [5, 2, 8, 1] through pass i = 0: min_idx starts at 0 (value 5).
- j = 1: arr[1]=2 > arr[0]=5 is False → min_idx stays 0.
- j = 2: arr[2]=8 > arr[0]=5 is True → min_idx becomes 2 (the LARGEST value so far).
- This confirms the loop is tracking the largest element, not the smallest — the fix is to flip the operator to '<'.

Optimization Applied

- Skip the swap entirely when min_idx == i (the current element is already the minimum), avoiding a redundant read/write on every pass where no movement is needed.

Corrected & Optimized Code

```
def selection_sort(arr):
    """
    Brute-force Selection Sort (ascending order).
    Repeatedly selects the minimum element of the unsorted
    suffix and places it at the boundary of the sorted prefix.
    """
    n = len(arr)
    for i in range(n - 1):              # last element is placed by elimination
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:    # FIXED: look for the minimum
```

```
                min_idx = j
        if min_idx != i:                # OPTIMIZATION: avoid a no-op swap
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Time Complexity Analysis

- Comparisons: always (n-1) + (n-2) + ... + 1 = n(n-1)/2 → $O(n^2)$ in best, average and worst case.
- Swaps: at most (n-1) in the worst case, but the optimization reduces swaps to 0 in the best case (already-sorted input), improving constant-factor performance without changing the asymptotic $O(n^2)$ bound.
- Space Complexity: O(1) — sorting is done in place.

Question 6. Debugging Bubble Sort Code *[Marks: 10]*

The given Bubble Sort implementation uses the wrong relational operator and performs a fixed number of full passes regardless of whether the array is already sorted.

Buggy Code (as given)

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n-1):
            if arr[j] < arr[j+1]:   # ERROR
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Errors Identified & Root Cause

- Comparison error: 'if arr[j] < arr[j+1]' swaps whenever the left element is SMALLER, which pushes large values to the front — this produces descending order, not ascending order.
- Efficiency issue: the inner loop always runs the full range(n-1) on every pass and there is no early-exit mechanism, so the algorithm performs O(n²) comparisons even on an already-sorted array.

Step-by-Step Debugging

- Trace arr = [4, 1, 3]: j=0 → arr[0]=4 < arr[1]=1 is False → no swap (4 should move right, but condition blocks it).
- j=1 → arr[1]=1 < arr[2]=3 is True → swap gives [4, 3, 1], which is moving away from ascending order.
- Flipping the operator to '>' makes larger neighbours move right-to-left correctly, i.e. the desired ascending bubble-up of the smallest values.

Optimization Applied

- Shrink the inner-loop range to n-i-1 since the last i elements are already settled after each pass.
- Introduce a 'swapped' flag to terminate early the moment a full pass makes no swaps (array already sorted).

Corrected & Optimized Code

```
def bubble_sort(arr):
    """
    Optimized Bubble Sort (ascending order) with early termination.
    """
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(0, n - i - 1):        # OPTIMIZATION: shrinking range
            if arr[j] > arr[j + 1]:           # FIXED: correct ascending condition
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:                       # OPTIMIZATION: early exit
            break
    return arr
```

Time Complexity Analysis

- Worst case (reverse-sorted input): $O(n^2)$ comparisons and swaps.
- Average case: $O(n^2)$.
- Best case (already sorted input): O(n) — thanks to the swapped flag, the algorithm exits after the first pass.
- Space Complexity: O(1) — in-place sorting.

Question 14. Debugging Binary Search Code (Infinite Loop Issue) *[Marks: 10]*

The given Binary Search implementation updates its boundaries in a way that never excludes the tested midpoint, so the search interval can stop shrinking and loop forever.

Buggy Code (as given)

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid          # ERROR
        else:
            high = mid         # ERROR
    return -1
```

Errors Identified & Root Cause

- Boundary update error: 'low = mid' and 'high = mid' both keep the already-tested midpoint inside the new search interval.
- Root cause: when low and high converge such that mid == low (integer division rounds down), the interval [low, high] does not shrink between iterations. Since mid is not excluded, the loop condition 'low <= high' remains true indefinitely, producing an infinite loop for keys that lie just above the midpoint in a 2-element interval.

Step-by-Step Debugging

- Trace arr = [1, 3], key = 3: low=0, high=1 → mid = 0.
- arr[0]=1 < key=3, so 'low = mid' sets low = 0 again — no change.
- The loop repeats forever with low=0, high=1, mid=0, never reaching index 1 where the key actually is.
- The fix is to always move strictly past the tested midpoint: low = mid + 1 when searching the right half, high = mid - 1 when searching the left half — this guarantees the interval shrinks every iteration.

Optimization Applied

- Use 'low + (high - low) // 2' instead of '(low + high) // 2' for the midpoint to avoid potential integer overflow on very large index ranges in languages with fixed-width integers (a defensive habit even though Python integers do not overflow).

Corrected & Optimized Code

```
def binary_search(arr, key):
    """
    Iterative Binary Search on a sorted array (ascending order).
    Returns the index of key, or -1 if not present.
    """
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = low + (high - low) // 2   # OPTIMIZATION: overflow-safe midpoint
```

```
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1               # FIXED: exclude mid, search right half
        else:
            high = mid - 1              # FIXED: exclude mid, search left half
    return -1
```

Time Complexity Analysis

- Best case: O(1) — key found at the first midpoint tested.
- Average / Worst case: O(log n) — the search interval halves on every iteration.
- Space Complexity: O(1) for the iterative version (O(log n) if implemented recursively, due to the call stack).

Question 37. Debugging String Matching Code (Incorrect Match Detection Condition)

The given brute-force string matching implementation decides success using the final value of the inner loop variable j after the loop ends, which is not a reliable signal of a complete match.

Buggy Code (as given)

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    result = []
    for i in range(n - m + 1):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        if j == m-1:      # ERROR
            result.append(i)
    return result
```

Errors Identified & Root Cause

- Unsafe success condition: 'j == m-1' only confirms that the LAST index the inner loop happened to reach was m-1 — it does not confirm that every character from j=0 to j=m-1 actually matched.
- Root cause: when a mismatch occurs on the final character of the pattern (j = m-1), 'break' still leaves j equal to m-1, so the buggy condition wrongly reports a match. Conversely, for a single-character pattern (m = 1) the inner loop only ever reaches j = 0 = m-1, so the code reports a 'match' at every position regardless of whether the characters actually agree — the loop variable is being reused as a correctness flag, which it was never designed to be.

Step-by-Step Debugging

- Trace text='abcXe', pattern='abcYe' at i=0: j=0,1,2 match; j=3 → 'X' != 'Y' → break with j=3.
- j == m-1 → 3 == 4 is False here, so this particular case happens to be safe.
- But trace text='a', pattern='b' at i=0: j=0 → 'a' != 'b' → break with j=0. Condition checks j == m-1 → 0 == 0 → True, so the code wrongly reports a match even though the characters differ.
- Fix: use an explicit boolean flag (or Python's for...else construct) that is only set to True after the inner loop completes all m iterations without breaking.

Optimization Applied

- Retain brute-force character-by-character comparison (as required by the CO2 scope) but exit the inner loop immediately on the first mismatch, avoiding unnecessary comparisons — this is already present and is preserved.

Corrected & Optimized Code

```
def string_match(text, pattern):
    """
    Brute-force substring search.
    Returns a list of all starting indices where pattern occurs in text.
    """
    n, m = len(text), len(pattern)
```

```
    result = []
    if m == 0 or m > n:
        return result
    for i in range(n - m + 1):
        matched = True                      # FIXED: explicit success flag
        for j in range(m):
            if text[i + j] != pattern[j]:
                matched = False
                break                        # OPTIMIZATION: stop at first mismatch
        if matched:
            result.append(i)
    return result
```

Time Complexity Analysis

- Worst case: O((n - m + 1) * m) ≈ O(n*m) — e.g. text = 'aaaa...a', pattern = 'aaab'.
- Best case: O(n) when most alignments fail on the first character comparison.
- Space Complexity: O(k) where k is the number of matches found (for the result list).

Question 79. Debugging Maximum–Minimum Finder Code (Extremes Initialized Incorrectly)

The given code initializes the minimum and maximum trackers to the wrong mathematical extremes, so neither variable can ever be updated.

Buggy Code (as given)

```
def min_max(arr):
    mn = float('-inf')   # ERROR
    mx = float('inf')    # ERROR
    for x in arr:
        if x < mn:
            mn = x
        if x > mx:
            mx = x
    return mn, mx
```

Errors Identified & Root Cause

- 'mn' is initialized to negative infinity, so the condition 'x < mn' can never be True for any finite value — mn is never updated and the function always returns -inf as the minimum.
- 'mx' is initialized to positive infinity, so 'x > mx' can never be True for any finite value — mx is never updated and the function always returns +inf as the maximum.
- Root cause: the two sentinel values are reversed. A minimum tracker must start at the LARGEST possible value it could be beaten by (e.g. +inf, or simply the first array element) so real data can lower it; a maximum tracker must start at the SMALLEST possible value (e.g. -inf, or the first array element) so real data can raise it.

Step-by-Step Debugging

- Trace arr = [3, 7, 1]: mn = -inf, mx = +inf initially.
- x=3: 3 < -inf → False; 3 > inf → False. Nothing updates.
- Same for x=7 and x=1 — the function returns (-inf, inf) regardless of input, which is meaningless.
- Fix: initialize both mn and mx from the first element of the array (arr[0]) after confirming the array is non-empty, then scan the remaining elements.

Optimization Applied

- Guard against the empty-list edge case explicitly instead of letting arr[0] raise an uncontrolled IndexError, improving robustness without changing the O(n) brute-force scan.
- Keep the two comparisons independent (if / if, not if / elif) so that on a 1-element array both mn and mx are correctly set, and so that a new minimum does not block a maximum check on the same element.

Corrected & Optimized Code

```
def min_max(arr):
    """
    Brute-force single-pass scan to find both the minimum
    and maximum values of a list, safe for empty input and
    for arrays containing only negative numbers.
    """
    if not arr:
        raise ValueError("min_max() received an empty list")

    mn = mx = arr[0]        # FIXED: seed both trackers with real data
    for x in arr[1:]:
        if x < mn:
            mn = x
        if x > mx:           # independent check, not elif
            mx = x
    return mn, mx
```

Time Complexity Analysis

- Best / Average / Worst case: O(n) — a single linear pass over the array is always required in the brute-force approach.
- Space Complexity: O(1) — only two scalar trackers are used.